COS 214 Practical 6 (TIVIDY)

06 October 2026

Aidan du Bruyn – 25040104
Connor Human – 22752154
Ishmael Ngwasheng – 24642542
Jay Macaskill – 25198387
Keval Govind – 24904407
Ruan van Tonder – 25284607
Thabo Ngwabi – 24742512
https://github.com/jaymacaskill/tividy

Task 1: Research Workflow Management Systems

A workflow describes the work needed to produce an outcome, who or what performs it, and the rules controlling its progress. A workflow management system stores these descriptions and manages their execution. A workflow definition describes the process, while a workflow instance is one execution of it. Each instance needs its own data, participants and progress. TIVIDY should therefore separate the reusable definition from information that changes during execution. Definitions typically contain activity descriptions, routing rules, required roles, and validation constraints, while instances contain the actual data values, assignments, progress, and history for that execution [1], [2].

Workflows involve more than a list of tasks. Activities may happen in sequence, run in parallel, or become available only when a condition is satisfied. Research identifies these as recurring workflow behaviours. BPMN, a standard for describing business processes, also represents individual tasks and smaller processes within larger ones [3]. These findings suggest that TIVIDY needs to represent grouped work and track the conditions that make each activity ready.

Routing determines which activity happens next. Assignment determines who performs it. Workflow systems typically model participants through roles or groups, with assignment rules determining which eligible participant receives a work item [2]. Camunda, an existing workflow-management platform, supports tasks assigned to a specific user or made available to eligible users or groups [4]. Approval, rejection and escalation also need defined rules. Escalation means passing work or a problem to another responsible person or level. TIVIDY should allow these policies to change without rewriting its basic task classes. Routing, assignment, approval and escalation rules are typically configurable rather than hard-coded, allowing organisations to change policies without modifying the workflow engine [1].

Workflow progress can also change when something happens, such as a task completing, a message arriving or a deadline passing. BPMN represents these occurrences as events [3]. Camunda's job workers perform automated tasks and report success or failure. Failed work can be retried, with a delay between attempts, before requiring intervention [5]. TIVIDY therefore needs clear responsibilities for events, external-system communication and failure handling. Integration with external systems is often handled through defined interfaces or connectors, so that changes to external services do not affect the core engine [5].

A workflow system must explain both its current progress and its past actions. An audit history records what happened during an instance. Camunda's Operate tool illustrates how users can inspect active and completed instances and investigate problems [7]. We propose that TIVIDY record actions and state changes, then use that information to show progress, overdue tasks and failures.

Changes to process definitions also need control. Camunda normally keeps existing instances on the definition version they started with [6]. TIVIDY should preserve that association. Recovery also has limits: cancelling unfinished work differs from reversing completed work. BPMN uses compensation for actions intended to reverse completed effects [3]. Restoring internal data alone cannot reverse every external action.

Together, these findings suggest a generic engine with separate responsibilities for definitions, instances, work items, participants, decisions, events and history. The selected scenario should supply its own activities, data and policies. Routing rules, assignment policies, approval and escalation logic, and external integrations are expected to vary independently from the core engine. These responsibilities must collaborate before the team makes final design pattern choices and designs the UML diagrams.

Bibliography

[1] Workflow Management Coalition, "Terminology & Glossary," WFMC-TC-1011, Issue 2.0, Jun. 1996. [Online]. Available: https://www.aiai.ed.ac.uk/project/wfmc/ARCHIVE/DOCS/glossary/glossary.html. [Accessed: Oct. 5, 2026].

[2] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros, "Workflow Patterns," Distributed and Parallel Databases, vol. 14, no. 1, pp. 5–51, 2003, doi: 10.1023/A:1022883727209. [Online]. Available: https://pure.tue.nl/ws/files/2053121/613310.pdf. [Accessed: Oct. 5, 2026].

[3] Object Management Group, "Business Process Model and Notation (BPMN), Version 2.0.2," OMG Document formal/2013-12-09, Dec. 2013. [Online]. Available: https://www.omg.org/spec/BPMN/2.0.2/PDF. [Accessed: Oct. 5, 2026].

[4] Camunda, "User tasks," Camunda 8 Documentation. [Online]. Available: https://docs.camunda.io/docs/components/modeler/bpmn/user-tasks/. [Accessed: Oct. 5, 2026].

[5] Camunda, "Job workers," Camunda 8 Documentation. [Online]. Available: https://docs.camunda.io/docs/components/concepts/job-workers/. [Accessed: Oct. 5, 2026].

[6] Camunda, “Versioning process definitions,” Camunda 8 Documentation. [Online]. Available: https://docs.camunda.io/docs/components/best-practices/operations/versioning-process-definitions/. [Accessed: Oct. 5, 2026].

[7] Camunda, “Introduction to Operate,” Camunda 8 Documentation. [Online]. Available: https://docs.camunda.io/docs/components/operate/operate-introduction/. [Accessed: Oct. 5, 2026].

Task 2: Define the Scenario & System

Proposed System Scope

TIVIDY will be a generic workflow management system that stores reusable process descriptions and manages their execution. It will coordinate work, apply rules, track progress and maintain a history of actions. The selected scenario will use these capabilities without determining the entire architecture. The scope should be revised as we continue our design.

TIVIDY will:

1. Manage workflow definitions. Store descriptions of activities, their relationships, required information and process rules. Check that a definition is valid before starting an execution.
2. Manage workflow instances. Create an independent execution from a definition. Track its progress and determine when it has completed, been cancelled or failed.
3. Manage activities and dependencies. Represent individual activities and groups of activities. Support sequential and parallel work, conditional routes and repeated work. An activity becomes available only when its required conditions are satisfied.
4. Control valid actions. Track each work item’s current state and allow only appropriate actions. Check that the participant requesting an action has permission to perform it.
5. Apply routing and assignment rules. Keep the decision about which activity happens next separate from the decision about who performs it. Allow these rules to change without rewriting the basic work-item classes.
6. Manage approvals and deadlines. Record approval outcomes, follow defined rejection or rework routes, and respond to missed deadlines. Apply escalation rules that transfer responsibility or request further intervention.
7. Handle events and external communication. Respond to relevant occurrences that affect progress. Communicate with external systems through defined interfaces that separate external-service details from the core engine.
8. Handle failures and maintain history. Distinguish expected business outcomes from technical failures. Apply configured retry or intervention rules. Record actions, outcomes and state changes without removing earlier history.
9. Provide monitoring and controlled recovery. Show current progress, waiting work, overdue activities and failures. Support reversal only where an action has a clearly defined recovery operation.

Information	Workflow Definition	Running Instance
Identity	Definition name, identifier and version	Instance identifier and the definition version being used
Work structure	Activity descriptions, groups and connections	Activities created for this execution and their progress
Data	Required fields and validation rules	Actual values, results and associated records
Participants	Required roles and assignment rules	Eligible participants and actual assignments
Decisions	Conditions controlling routes and approvals	Decisions made and their outcomes
Timing and recovery	Deadline rules, retry limits and recovery options	Actual deadlines, attempts and failures
History	Descriptions of permitted activities and transitions	Recorded actions, actors, times and state changes

TIVIDY will coordinate activities and record their progress. Human participants and external systems will perform the work assigned to them. Process-specific calculations and rules will remain in activity implementations, definitions or policies.

The initial scope will include basic participant and role information, external-system interfaces and progress reporting. We propose deferring a graphical workflow editor, full BPMN compatibility, distributed execution, production identity services, advanced analytics and automatic transfer of running instances between definition versions.

Each instance must keep its own data, assignments, progress and history. Each work item must have a clear owner responsible for its creation and lifetime. Dependencies must be satisfied before work proceeds, and parallel activities must meet the agreed completion conditions before subsequent work begins.

Repeated or outdated events must not cause incorrect changes. Recovery actions must preserve the original history and create a record of the recovery performed.

A work item may be created, made available, assigned, started, completed, rejected, cancelled or escalated. Operations which are valid in one state may be invalid in another, so TIVIDY must enforce valid transitions. Work items may acquire additional behaviour (such as validation, priority handling, auditing, notifications, security, etc.) without becoming a different kind of task.

Each workflow instance remains associated with the definition version it was created from Changes to a definition does not affect work already in progress. Running structures (instances, stages, work items, etc.) can be traversed for different purposes (such as monitoring, reporting, recovery) without exposing their internal storage to the rest of the system.

Design Rationale

TIVIDY is designed as a generic workflow management system, the design must support many different processes without being completely rewritten for each one. Our research identified recurring responsibilities (definitions, instances, work items, routing, assignment, events, history, external integration) and showed that these responsibilities vary independently in real systems. The design rationale explains why the system is structured the way it is, based on research and the proposed scope.

Separating Workflow Definitions From Running Instances

The separation between a workflow definition and a workflow instance established in the research phase of this project is the foundation of the design since it allows one definition to produce many independent instances without duplication. Each instance owns its own data, assignments, progress and history, so changes to one running instance cannot affect another. This is why the scope iterates that instances are created as independent executions, and why the information table separated definition-level information (activity descriptions, routing rules, required roles) from instance-level information (actual data values, assignments, progress, history)

Keeping Routing And Assignment Separate

Our research further showed that routing (which activity happens next) and assignment (who performs it) are distinct concerns, and that both are typically configurable rather than hard-coded. Our main example, Camunda, supports tasks assigned to specific users or made available to eligible users or groups, and organisations can change these policies without modifying the engine. The design keeps routing and assignment rules outside the basic work-item classes, so that policies can change independently of the work itself. Rules are changeable without rewriting the basic work-item classes.

Modelling The Work-Item Lifecycle Explicitly

The research also identified a recurring set of states that a work item passes through (e.g. created, available, assigned, started, completed, rejected, cancelled, escalated) and showed that operations valid in one state may not be valid in another. The design models the lifecycle explicitly, so that the engine can enforce valid transitions and prevent invalid actions. We include controlling valid actions and checking participant permissions. The lifecycle is not just a label, it determines what can happen next.

Handling Events And Triggers Separately From Core Work

Workflow progress can change when something happens (a task completes, a message arrives or a deadline passes). These are modelled as events. Multiple interested parts of a system may need to react to the same event, and external systems are often integrated through defined interfaces or connectors so that changes to external services do not affect the core engine. The design separates event handling from the core workflow model, such that new rules, interested parties or integrations can be introduced without redesigning the engine. Event handling, external communication and failure handling are therefore separate responsibilities.

Supporting Added Behaviour Without Subclassing

A work item may acquire additional behaviour (e.g. validation, priority handling, auditing, notifications, security) without becoming a different kind of task. Hard-coding these behaviours would make the design rigid and force duplication whenever a new combination is needed. We therefore allow behaviour to be added to work items independently of their core type, so that the same task can be validated, prioritised, audited or secured as needed.

Recording History Independently Of The Core Model

A workflow system must be able to explain its current progress and its past actions. History and audit records are essential for monitoring, reporting and recovery. The design treats history as a separate responsibility, recording actions, outcomes and state changes without removing earlier records. History is separated from the core work-item model and recovery actions are required to preserve the original history.

Keeping Definition Versions Stable For Running Instances

Changes to process definitions need control (e.g. Camunda normally keeps existing instances on the definition version they started with). We preserve this association, such that a change to a definition does not affect work already in progress. The information table includes the definition version in the instance's identity. Versioning is part of managing workflow definitions.

Designing For Independent Variation

These decisions identify the parts of TIVIDY expected to vary independently (routing rules, assignment policies, approval and escalation logic, event handling, external integrations, behaviour attached to individual work items). The design keeps these separate from the core engine so that the system remains reusable across different processes. Our elected scenario will, however, supply its own activities, data and policies, but will not determine the architecture of the engine itself.

Realistic Workflow Scenario: Employee Onboarding, Employee Services

This scenario is executed using TIVIDY as a generic workflow engine. The onboarding process is stored as a workflow definition, each new hire creates an independent workflow instance, and the roles, decisions, dependencies and failures below are modelled using TIVIDY's work items, routing rules, events and history.

The scenario begins when a candidate accepts an offer and ends when the new hire is fully onboarded. This process consists of a lot of setup, check-ups and paperwork across various departments and through many different employees within the workplace.

Organisation or environment

Companies can have hundreds of employees across numerous branches or offices, hiring multiple employees each year. The number of hires is not constant and can differentiate in volume throughout the year. Different companies and positions have varying types of policies for employment - the laws of the country also have an impact on these policies. Policies may change over time, so the onboarding process for a new hire may differ from that of a previous employee.

Process being managed

If a prospective candidate accepts an offer to work as an employee, the company must do some initial paperwork. The required equipment must be acquired and the necessary access to said equipment must be granted. The company must perform some compliance checks and then the candidate must be welcomed to the company, followed by a training course to allow the candidate to be integrated as an employee. Lastly after said training the employee should be checked to ensure they successfully learnt how to be a productive employee.

Participants and roles involved

Role	Responsibility
New Hire	Completing and submitting forms, sending documents, completing training and attending sessions and seminars
Hiring Manager	Conducts performance reviews, explaining the requirements of the role
HR Coordinator	Verify documentation and issue contracts

IT Technician	Requesting new hardware, creating necessary accounts and giving access
Payroll Officer	Sets up the payroll and employee benefits
Mentor	Fellow employee to assist new hire to adapt to new workplace
HR Head	Handles any escalations encountered
Administrator (TIVIDY-level role)	Manages workflow and monitors the entire system
Legal Advisor	Work checks and manages contracts
Security Officer	Runs background checks and approves access privileges
Facilities Coordinator	Desk and parking space, gives building pass
Auditor (TIVIDY-level role)	Checks compliance evidence

Administrator and Auditor are TIVIDY-level roles rather than onboarding participants. The Administrator manages the workflow engine and monitors running instances, whilst the Auditor inspects compliance evidence recorded in the workflow history.

Major pieces of work

1. Pre-boarding: Create and submit the contract, candidate signs and inputs personal details. Identity and bank documents are sent and then verified.
2. Compliance checks: Background and conflict of interest checks

3. Provisions: Shipping and preparing of equipment. Account creation and assigning access privileges. Set up required facilities (desk and parking), payroll and benefits. Once all completed, sign off on all the provisions.
4. First week: Welcome new hire. Assign a mentor and do required training (code of conduct, health and safety).
5. Training: Assign tasks and review the hire's work. At the end give a decision of their work
6. Completion: Sign off employee and let new hire join the ongoing employee lifecycle

Several pieces of work are themselves sub-processes. IT setup includes account creation, hardware provisioning and access assignment. Training includes multiple modules that must each be completed.

Important decisions and routing points

Probation outcome: Outcomes include confirm, extend or terminate. A second extension request escalates to the HR Head.
Document verification outcome: Outcomes include accepted, rejected or escalated to Legal Advisor.
Mentor assignment: Depends on workload, skills and current role, dependent on department.
Employment type: Determine the hire's path, dependent on future role. For example interns may not have employee benefits.
Provision sign-off: Only occurs when all provisions are provided

Approvals and dependencies

Provisions depend on contracts being signed and documents being verified while equipment is dispatched only if address is confirmed. Privilege access is dependent on the Security Officers approval. Payroll set up requires verified bank details. Escalations require multi level approvals such as Hiring Manager and HR Head. Routing decisions have data dependencies such as location, start date, department, required seniority, employment type and access level.

Possible failures or exceptions

Situation	How to Resolve
New hire does not submit required documents in time	Give a reminder, then notify the Hiring Manager and if necessary escalate to the HR Head
Documentation rejected	Explain the reason to the hire
Failed background check	Process paused and Legal Advisor and HR Coordinator are notified
Change in start dates	Deadlines are recalculated

Role changes	Workflow is re-routed. Provisions are changed as well as necessary access privileges
Candidate rescinds offer or new hire withdraws mid-training	Tasks cancel, access privileges are revoked
Policy changes mid-training	Current hires continue, however new hires follow new policy
Approval incorrectly assigned	Undo last action

External systems

Email and calendar services for deadlines and reminders.
Background check for screening processes.
Building access system.
Training system to track task assignment and completion.
Analytics, to track workflow

Expected outcome of workflow

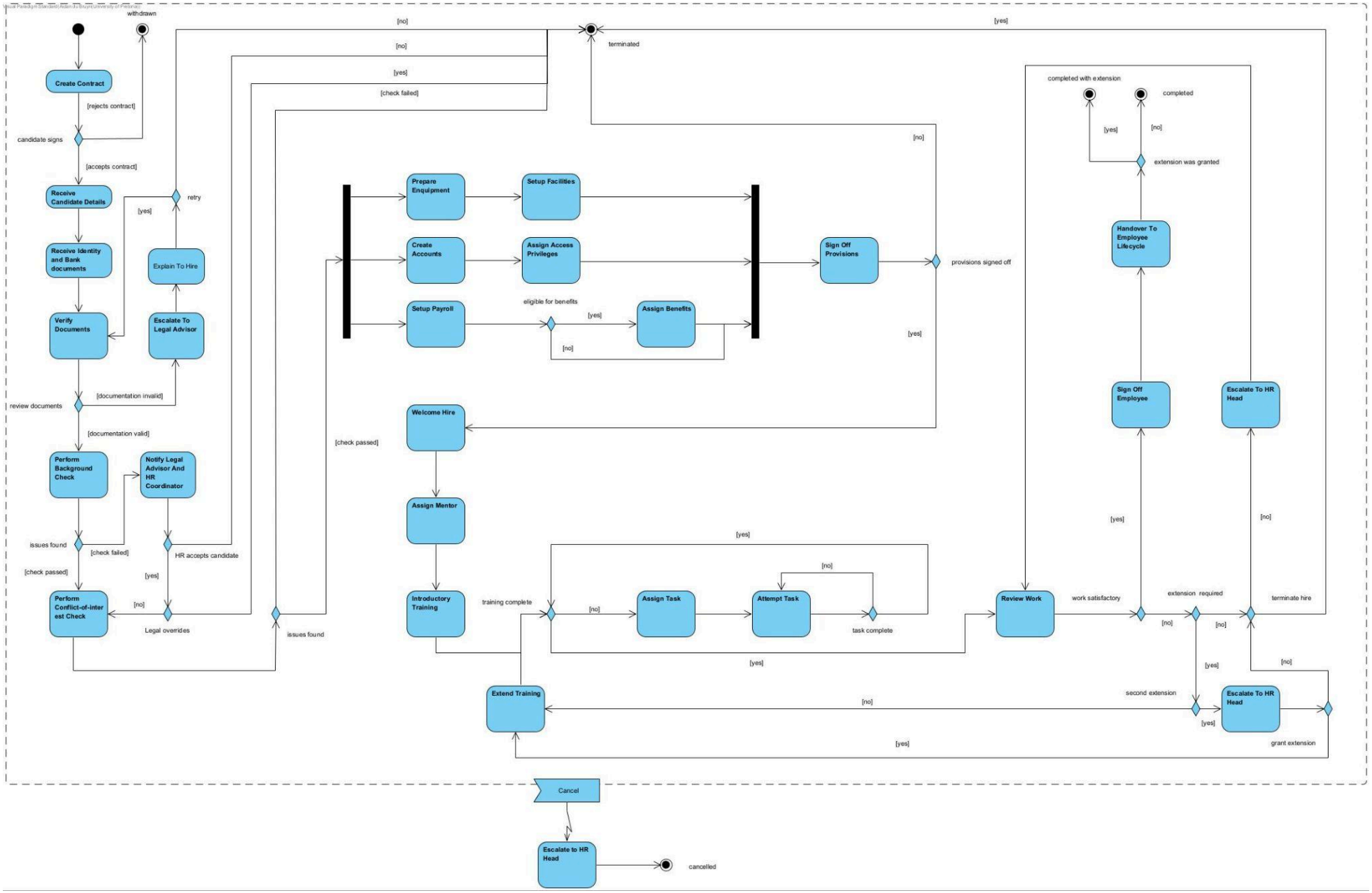
Every onboarding instances ends in one of the following states:

- Completed (probation confirmed)
- Completed with extension
- Terminated (during probation)
- Withdrawn (before process starts)
- Cancelled (during probation)

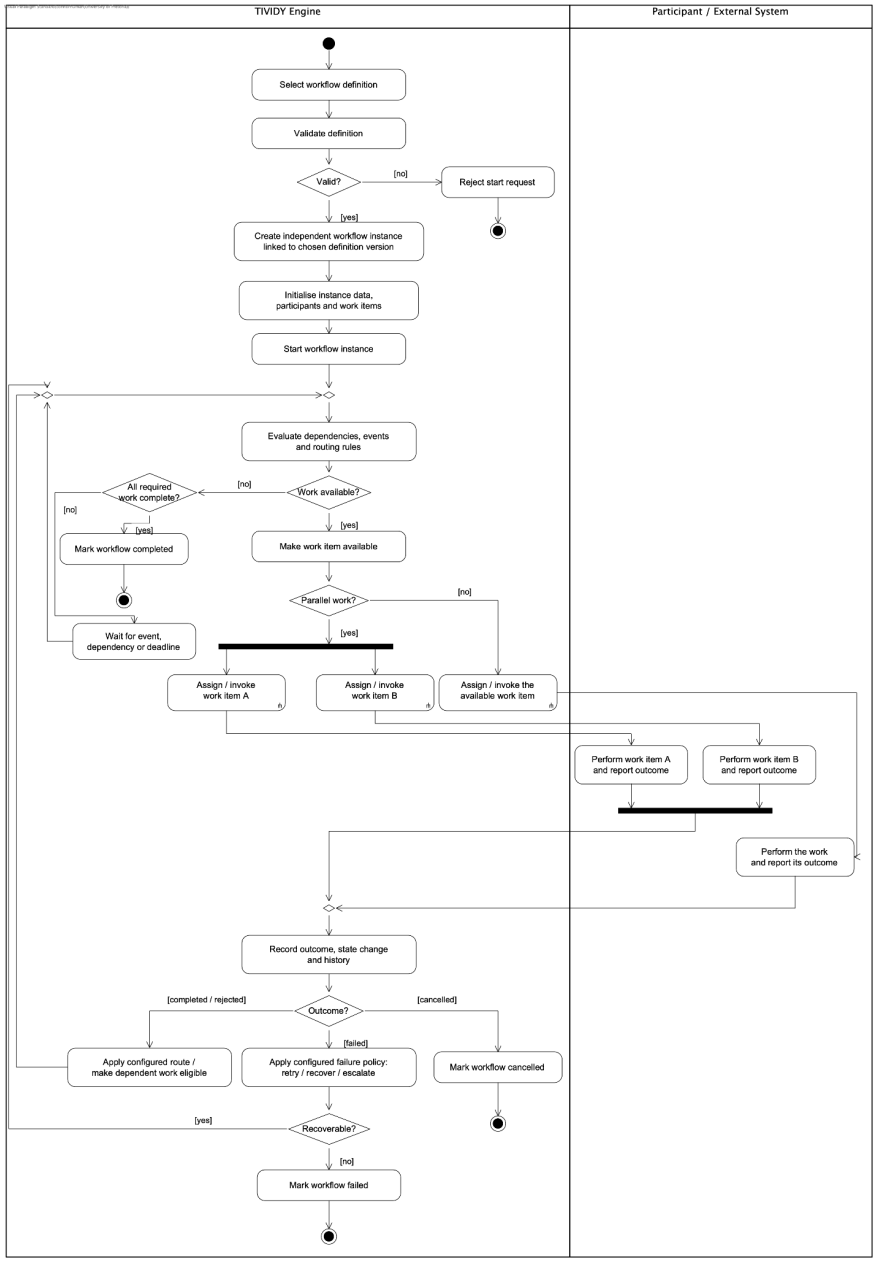
The system will keep record of the different roles duties, what was or was not completed when and by who. This includes any state changes and escalations that occur. The system will produce an employee with the proper accounts, payroll and training. Relevant notifications will be displayed.

Task 3: UML Activity Diagrams

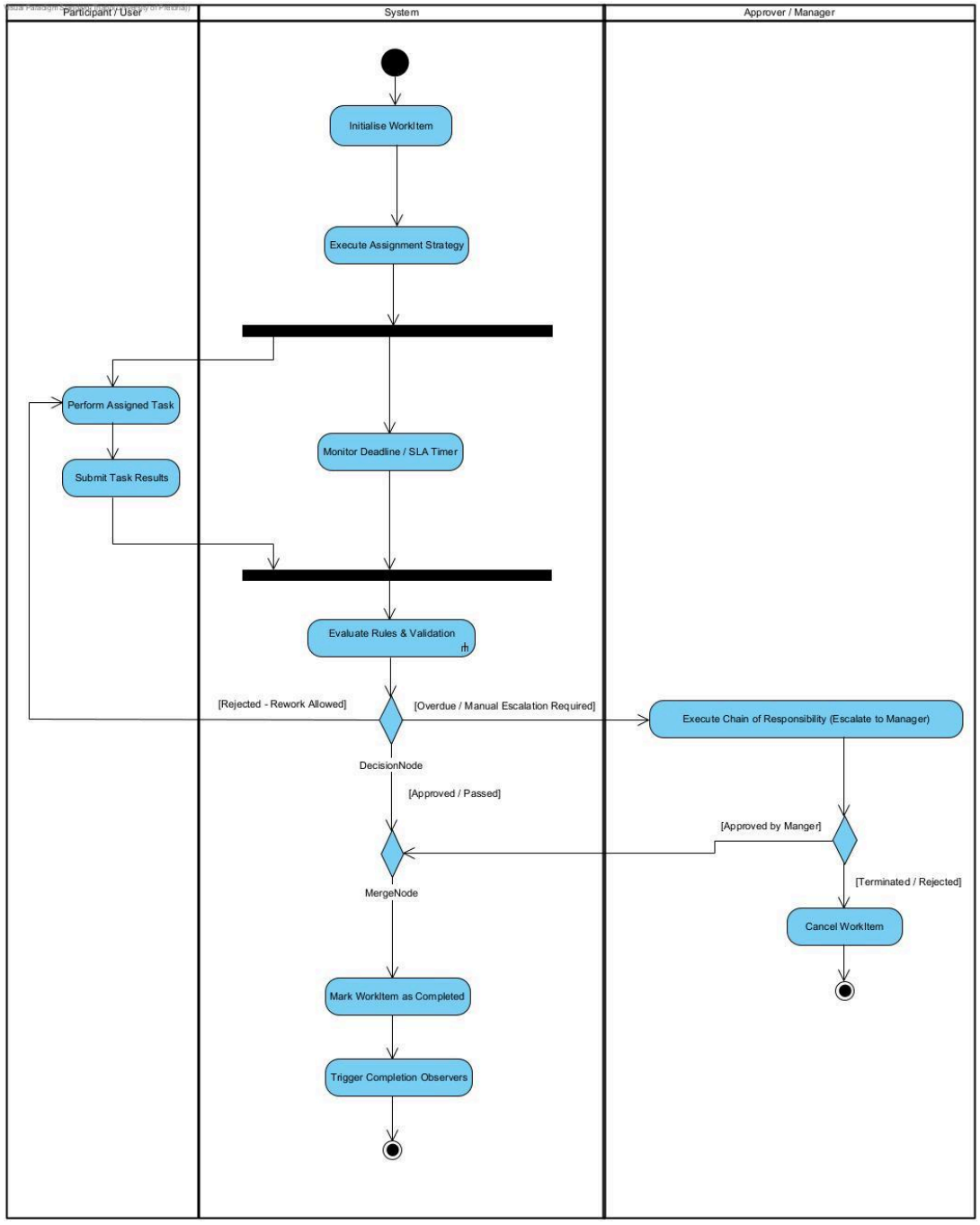
Scenario Specific Activity Diagram



Generic Activity Diagram 1: Workflow Execution



Generic Activity Diagram 2: Work Assignment and Escalation Workflow



Task 4: Design Patterns

Pattern	Category	Design problem in TIVIDY	GoF participants	Collaboration	What varies	Why appropriate	Diagram evidence
Composite	Structural	Onboarding work is nested (IT setup, Training modules, Provisions stage). Callers must treat one task and a whole stage identically without knowing how children are stored.	Component: WorkItem. Leaf: TaskItem†. Composite: StageItem† (children held by unique_ptr). Client: WorkflowInstance.	WorkflowInstance holds one root WorkItem. StageItem forwards makeAvailable(), cancel() and accept() to its children and reports complete only when its completion condition holds.	Tree shape per definition (Provisions has 3 parallel branches, Training has N modules) and each stage's completion rule.	Children can be added or reordered in the definition without any caller changing. Ownership is explicit: instance owns the tree, tree owns its items (composition).	Seq 1 note
Decorator	Structural	The same task needs different extras (Verify Documents: validation + security; Grant Access: security + audit; overdue item: priority). Subclassing gives 2^n combinations.	Component: WorkItem. ConcreteComponent: TaskItem, StageItem. Decorator: WorkItemDecorator†. ConcreteDecorators: SecureWorkItem, ValidatedWorkItem†, AuditedWorkItem†, PriorityWorkItem†.	Each decorator holds a WorkItem* and implements the same interface. SecureWorkItem.complete(outcome) calls checkPermission(user) then delegates (Seq 2, steps 1-4). Decorators stack: Secure(Audited(TaskItem)).	Which concerns wrap which task, chosen per definition.	Security, audit and priority are optional and combinable per task, so they must not become task subclasses (spec: a task may acquire behaviour without becoming a different kind of task).	Seq 2

Adapter	Structural	Email/calendar, background-check provider, building-access and training systems each have their own interface; the engine must not depend on them.	Target: ExternalConnector† (send(), query()). Adapters: EmailServiceAdapter, BackgroundCheckAdapter†, BuildingAccessAdapter†, TrainingSystemAdapter†. Adaptees: vendor APIs.	NotificationObserver calls sendEmail(hire, "Welcome") on the connector interface (Seq 1, step 22). The adapter translates to the vendor request format and maps the response or failure back.	Vendor, protocol and payload format.	Switching email provider or adding a second background-check vendor changes one adapter. Technical-failure retry sits behind this interface, separate from business outcomes.	Seq 1
Facade	Structural	Starting a workflow touches the library, validation, instance creation, history and events. Clients such as the Administrator should not orchestrate these.	Facade: Tividy. Subsystems: WorkflowLibrary, WorkflowDefinition, WorkflowInstance, ActionHistory, EventPublisher.	startWorkflow("Onboarding", hireData) does lookup, checkIsValid(), createInstance(), start() and returns an instanceId (Seq 1, steps 1-23).	Subsystem internals, which can change without affecting callers.	One stable API for the Administrator and Auditor roles; keeps subsystems decoupled from each other.	Seq 1
Prototype	Creational	One onboarding definition creates many hire instances. Each needs its own copy of the work tree, data, assignments and history; editing one must never alter	Prototype: WorkItem with clone() : unique_ptr<WorkItem>. ConcretePrototypes: TaskItem, StageItem (deep-copies children), WorkItemDecorator subclasses (clone	WorkflowDefinition owns a template tree built by WorkflowBuilder. createInstance clones it, injects hireData, and passes the clone and version number to WorkflowInstance	The template's structure and which concrete types it contains, with no change to instance-creation code.	Cloning copies structure including decorators without the engine knowing concrete classes; enforces definition/instance separation. Risk: every clone() must	Seq 1 steps 8-11 (new)

		another or the definition.	wrapped item, then re-wrap). Client: WorkflowDefinition.createInstance(hireData).	(Seq 1, steps 8-11). The template is never executed.		deep-copy, never share pointers.	
Builder	Creational	An onboarding definition is a deep tree with dependencies, role requirements, deadlines and decorators. It must be assembled step by step and validated before use.	Builder: WorkflowBuilder†. ConcreteBuilder: OnboardingDefinitionBuilder†. Director (optional): WorkflowLibrary loader. Product: WorkflowDefinition.	Builder calls addStage("Provisions"), addTask(...), addDependency(...), setAssignment(...), build(). build() returns a definition whose checkIsValid() is later called by Tividy.	The process being defined (onboarding now, procurement later) and the construction steps.	A single constructor cannot express this structure. Keeps the scenario's workflow out of the engine: only the builder knows onboarding.	Seq 1 note
State	Behavioural	Valid operations depend on state: assign() is invalid on a Completed item; reject() behaves differently when canRetry holds. if/else chains on state are forbidden (Rule 6).	Context: WorkItem. State: WorkItemState. ConcreteStates: CreatedState†, AvailableState†, AssignedState†, InProgressState†, UnderReviewState†, EscalatedState†, CompletedState†, CancelledState†.	WorkItem.complete() asks state->canComplete() (Seq 2, steps 5-6), then calls moveToCompleted() which calls setState(Completed). Illegal calls raise InvalidStateError. Transitions follow the State Diagram, e.g. reject()[canRetry] returns to InProgress.	Which actions each state permits and the next state.	The lifecycle determines what can happen next. A new state (e.g. OnHold for a failed background check) is one new class.	Seq 2, State diagram

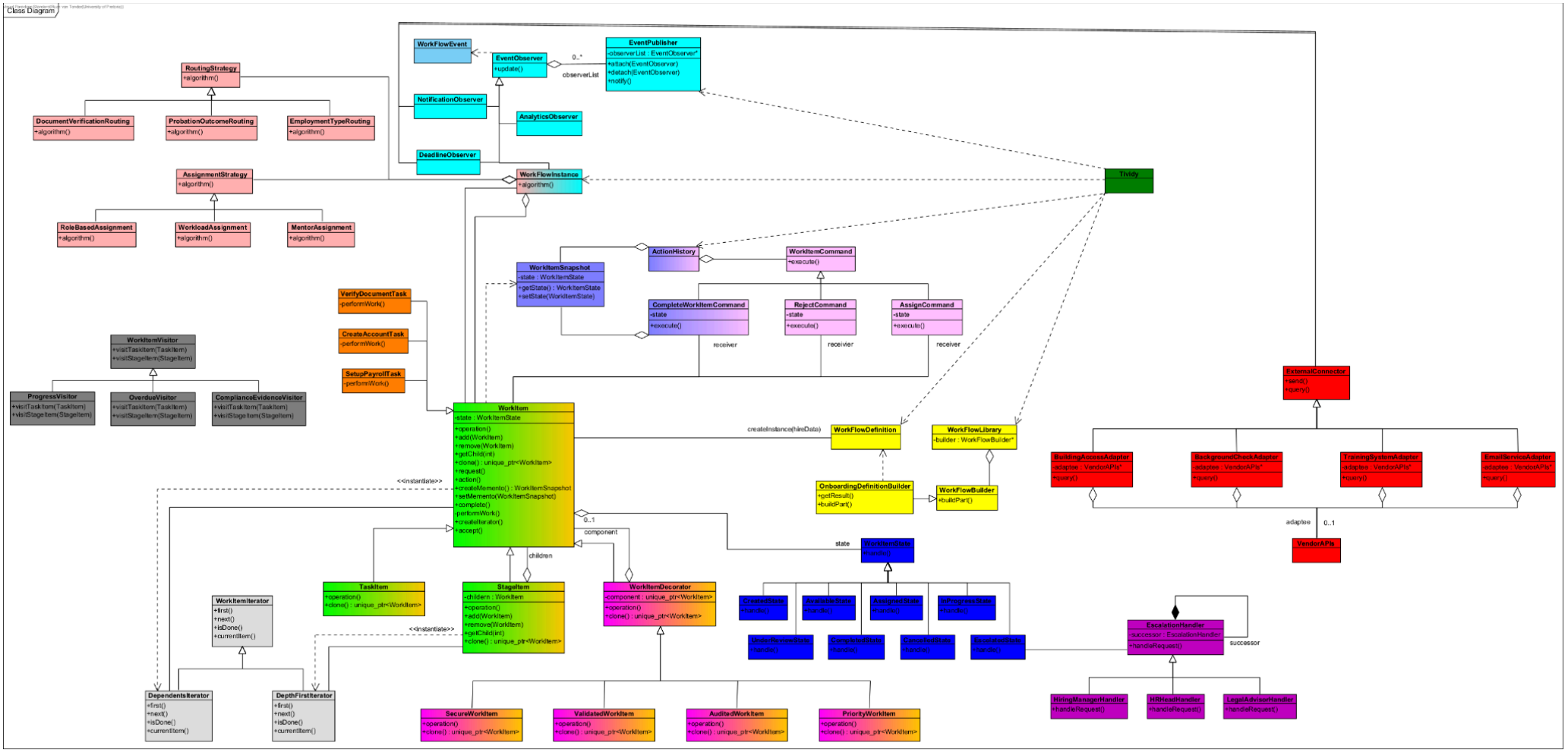
Strategy	Behavioural	"What happens next" (routing) and "who does it" (assignment) are separate decisions that vary by organisation and stay out of work-item classes. Two strategy families.	Context: WorkflowInstance. Strategy A: RoutingStrategy.chooseNextItems(item, outcome); concretes DocumentVerificationRouting†, ProbationOutcomeRouting†, EmploymentTypeRouting†. Strategy B: AssignmentStrategy.chooseParticipant(item); concretes RoleBasedAssignment†, WorkloadAssignment†, MentorAssignment†.	After completion WorkflowInstance calls RoutingStrategy.chooseNextItems (accepted/rejected/escalated give different successors, Seq 2 steps 19-22). For each next item it calls AssignmentStrategy.chooseParticipant (steps 23-25).	Routing: document accepted/rejected/escalated to Legal Advisor; probation confirm/extend/terminate; intern vs employee benefits path. Assignment: mentor by workload, skills, department; IT technician by role.	Policies change without editing WorkItem (scope rule; Camunda's configurable assignment). Two interfaces because the scenario varies the two choices independently.	Seq 2, Activity 2
Observer	Behavioural	One occurrence (e.g. WorkItemCompleted) must trigger several unrelated reactions: route next work, notify people, write analytics, update the training system. The completing item	Subject: EventPublisher. Observer: EventObserver. ConcreteObservers: WorkflowInstance (routing), NotificationObserver, AnalyticsObserver†, DeadlineObserver†.	WorkItem publishes WorkItemCompleted (Seq 2, steps 15-16). EventPublisher calls onEvent on each subscribed observer. Workflow-start event drives the welcome email (Seq 1, steps 20-22).	The set of observers and which events each subscribes to.	A new interested party (e.g. Facilities) is a new subscriber with no engine edit. Events carry an id so repeated events are ignored, as the scope requires.	Seq 1, Seq 2

		must know none of them.	Event: WorkflowEvent†.				
Command	Behavioural	The Auditor must see who did what and when, and "approval incorrectly assigned" must be undoable. Actions need to be objects, not lost method calls.	Command: WorkItemCommand †. ConcreteCommands: CompleteWorkItemCommand, AssignCommand†, RejectCommand†. Receiver: WorkItem. Invoker/history: ActionHistory.	WorkItem builds CompleteWorkItemCommand(item, outcome, snapshot) and calls ActionHistory.record (command, user, time) (Seq 2, steps 13-14). undo() restores the snapshot and logs a compensating record rather than deleting history.	The set of auditable actions and each action's reversal.	History is append-only as required. Commands without a defined undo() (e.g. email already sent) are marked non-reversible, matching the research point that internal restoration cannot undo external effects.	Seq 2
Memento	Behavioural	Undo needs the item's pre-change state, but ActionHistory must not see the item's private fields.	Originator: WorkItem. Memento: WorkItemSnapshot. Caretaker: CompleteWorkItemCommand (ActionHistory as holder).	WorkItem.saveSnapshot() runs before the change (Seq 2, step 9). The command stores the snapshot. On undo, WorkItem.restore(snapshot) resets state and assignment.	Which fields a snapshot captures per item type.	Supports "Undo last action" without breaking encapsulation. Snapshot covers internal state only (state, assignee, outcome); external effects are handled by Command's compensation flag.	Seq 2

Template Method	Behavioural	Every task must be completed in the same order: check state, snapshot, do task-specific work, transition, record, publish. Only the work differs.	AbstractClass: WorkItem with complete(outcome) as template. Primitive operation: performWork(outcome) (pure virtual). ConcreteClasses: VerifyDocumentsTask†, CreateAccountTask†, SetupPayrollTask† (subclasses of TaskItem).	complete() runs Seq 2 steps 5-15 and calls performWork() at step 10.	Only the work step.	Recording and event steps cannot be skipped by a new task type, so the audit trail is guaranteed.	Seq 2 note
Iterator	Behavioural	Routing needs the items waiting on a completed item; monitoring needs all items. Callers must not know whether the tree uses vectors or maps.	Iterator: WorkItemIterator. ConcreteIterators: DependentsIterator† (items waiting on X), DepthFirstIterator†. Aggregate: WorkItem/StageItem with createIterator().	WorkflowInstance calls getDependents(item) (Seq 2, steps 17-18) and receives an iterator.	Traversal order and filtering, and the underlying storage.	Spec requires traversal without exposing internal storage. Dependents and full-tree walks are different traversals over the same composite.	Seq 2
Visitor	Behavioural	Administrator needs progress and overdue reports; Auditor needs a compliance-evidence listing. Putting report(), overdue() and exportAudit() in	Visitor: WorkItemVisitor† (visitTask(TaskItem&), visitStage(StageItem&)). ConcreteVisitors: ProgressVisitor†,	WorkflowInstance.accept(v) passes v to the root. StageItem.accept calls v.visitStage(*this) then forwards to children. Decorators	The reports and views: new operations over a stable set of item types.	Item hierarchy is stable (task, stage) while reports keep growing. A new report is one new class; the core model is untouched.	New (no diagram yet)

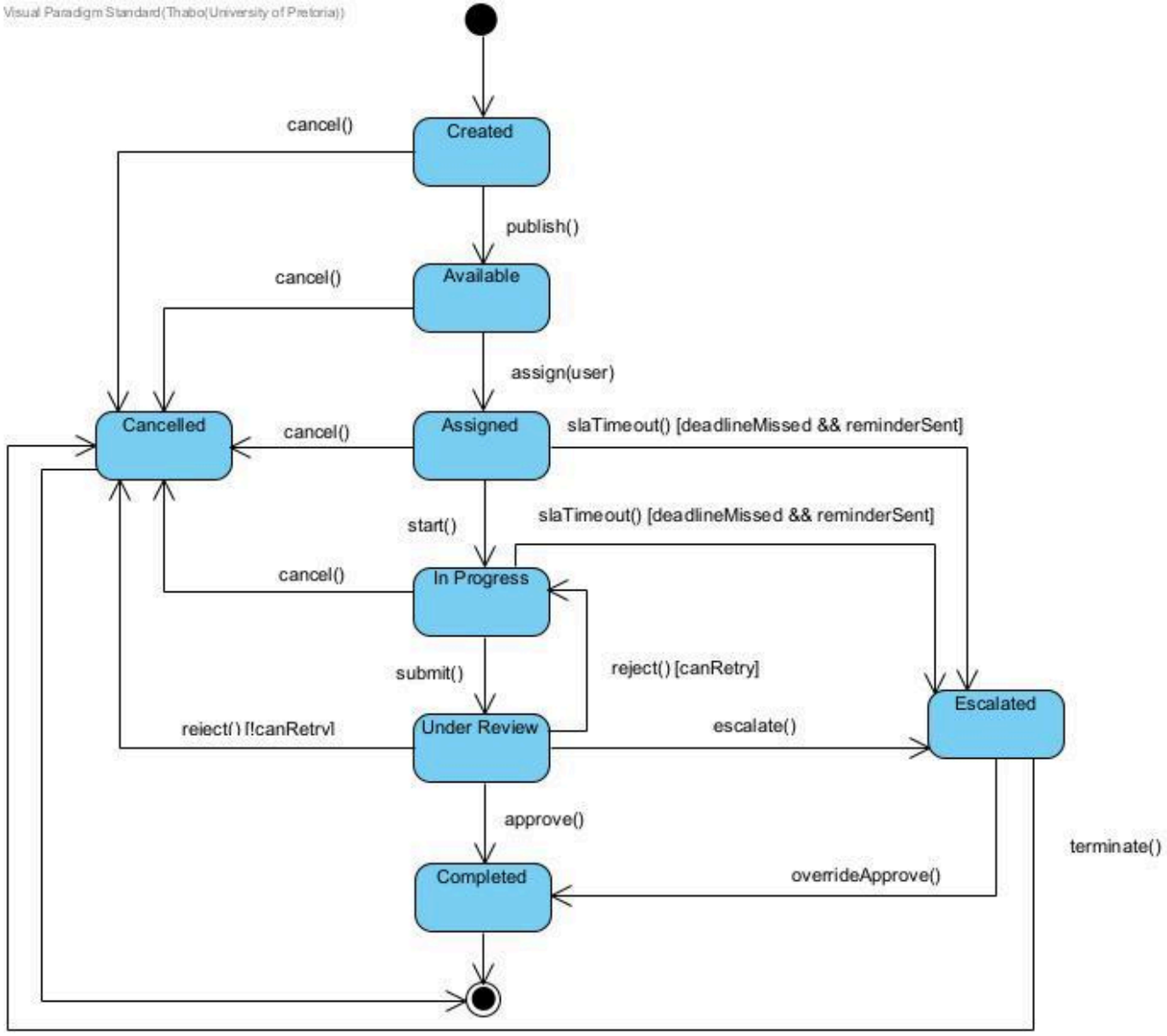
		every WorkItem would bloat the core model and need edits for each new report.	OverdueVisitor, ComplianceEvidenceVisitor†. Element: WorkItem.accept(WorkItemVisitor&). ConcreteElements: TaskItem, StageItem. ObjectStructure: WorkflowInstance.	forward accept to the wrapped item. OverdueVisitor collects items with a deadline before now and state not Completed/Cancelled.			
Chain of Responsibility	Behavioural	An unanswered or rejected item escalates upward: Hiring Manager, then HR Head. A second probation extension goes straight to HR Head. The requester must not know who will decide.	Handler: EscalationHandler†. ConcreteHandlers: HiringManagerHandler†, HRHeadHandler†, LegalAdvisorHandler† (document disputes). Client: EscalatedState / WorkItem.	Item calls handler->handle(request). A handler approves, rejects, or passes to its next. Decision feeds the merge in Activity Diagram 2: approved -> Completed (overrideApprove()), rejected -> Cancelled.	Chain order and length per organisation; per-request thresholds such as extension count.	Multi-level approval is configured, not coded. A new level (e.g. Regional Director) is a new handler.	Activity 2

Task 5: UML Class Diagram

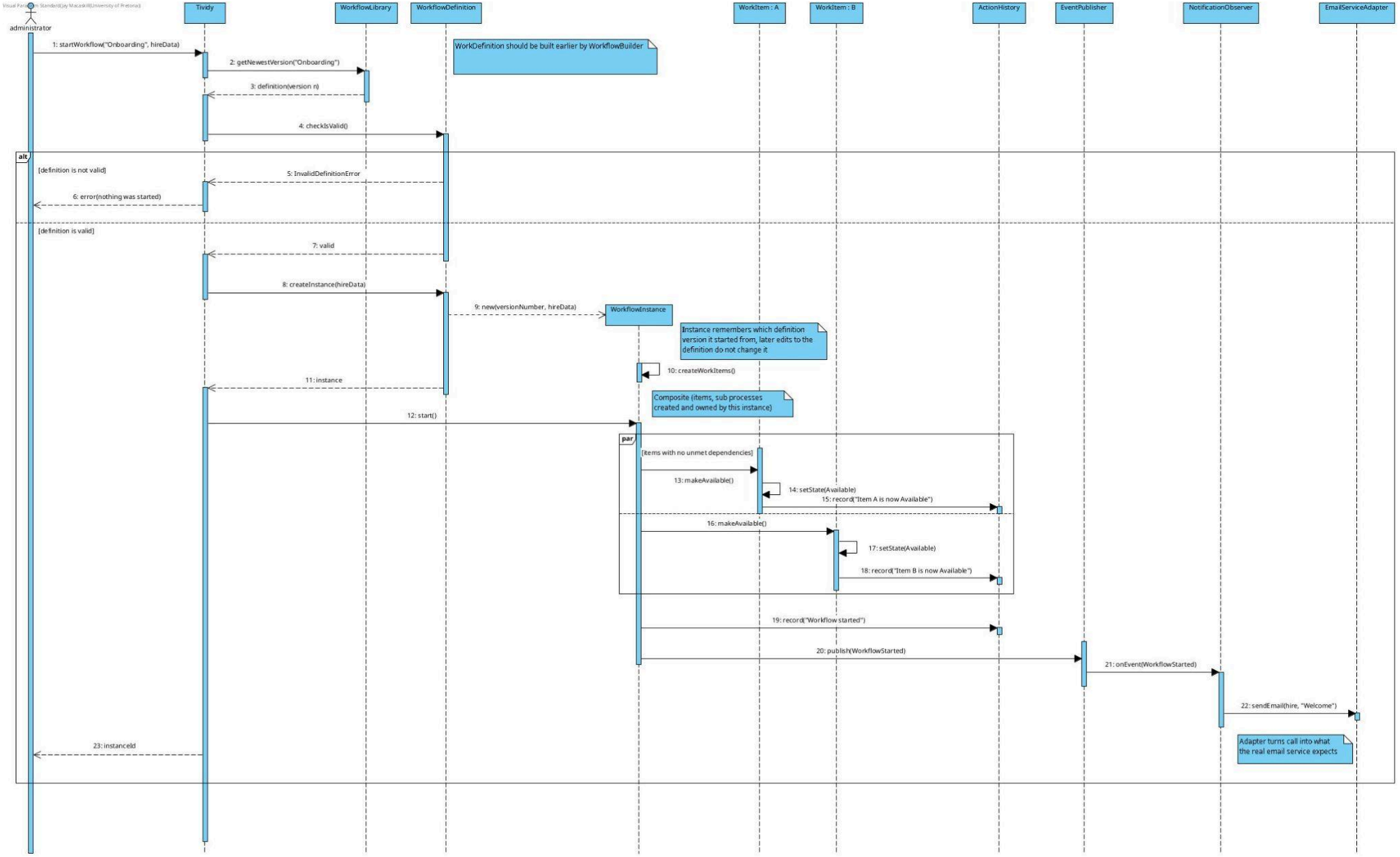


Task 6: Runtime Behaviour Diagrams

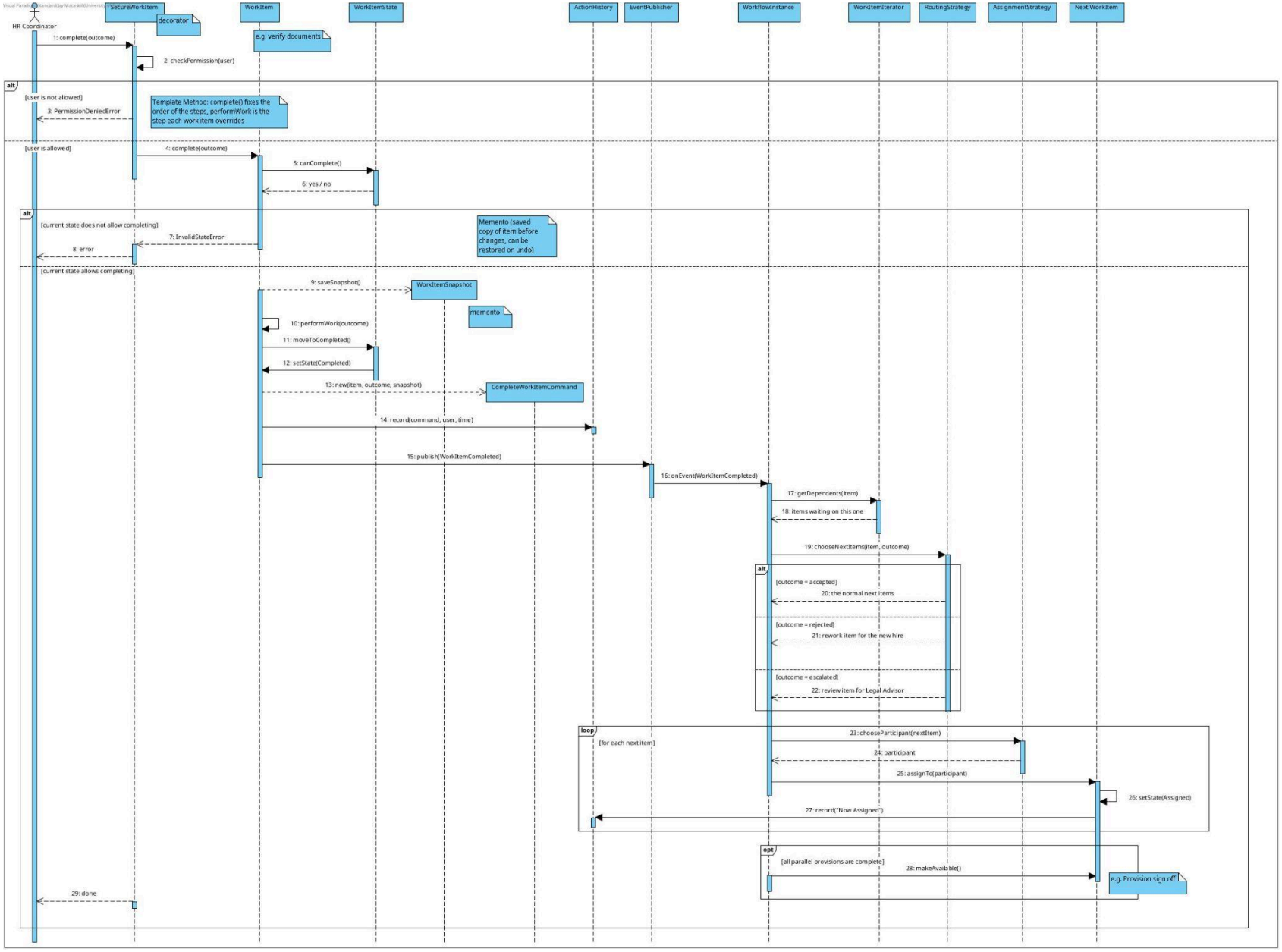
State Diagram: WorkItem Lifecycle State Diagram



Sequence Diagram 1: Creating & Starting a Workflow



Sequence Diagram 2: Complete a Work Item & Decide What Happens Next



Task 7: Design Decisions & Revisions

Important Design Decisions

Table of Revisions

Date of Revision	Problem / Question Being Considered	Original Design / Assumption	Alternatives Considered	Change Made	Why Change Was Made	Updates to Relevant Diagrams
6 Oct	**Pattern Mapping**: pattern set grew from 12 to 15	Prototype, Visitor and Chain of Responsibility were not identified	N/A	Added them to the pattern table	Prototype for cloning a definition into instances, Visitor for reports, Chain of Responsibility for escalation	Pattern table was updated
6 Oct	**UML Diagrams**: known inconsistencies	Diagrams have been drawn conceptually from the scenario description before the engine exists, so there are inconsistencies across diagrams, but not across the concepts modelled by the diagrams	N/A	For future revisions based on an existing engine, we will fix the inconsistencies	The engine will settle the final names and inconsistencies, solving the constant changing of diagrams based on naming conventions of a conceptual, incomplete model open to future changes	Yes, once the engine is finalised diagrams will need to be updated to reflect this
12 Oct	**Git & GitHub**: GitHub Workflow	Was not initially established	N/A	A GitHub workflow has been established with automated checks, updated rulesets, etc. after first progress	This makes collaboration more efficient, rules are strictly enforced and the repository is primarily self-managed	N/A

				meeting with the TA		

Task 8: GitHub WorkFlow

Repository Structure

The TIVIDY project is hosted in a single GitHub repository that serves as both the development workspace and the evidence of the team's engineering process. The repository is organised so that code, tests, documentation and design artefacts live together and are reviewed through the same pipeline.

```
.
├── src/
├── include/
├── tests/
├── app/
├── docs/
│   └── img/
│   └──research/
│   └── archive/
├── .github/
│   ├── workflows/
│   └── pull_request_template.md
│   └── CONTRIBUTING.md
├── README.md
├── Makefile
├── .gitignore
└── report/
```

The docs/ directory holds the research findings and references, diagrams and our PA6 document before beginning the implementation of TIVIDY. The repository records reflect the final artefact and the reasoning and processing that produced it.

Branching Strategy

main is protected. No member may push to it directly. Every change reaches main through a pull request opened from a branch whose name follows a fixed convention:

prefix/short-description

Where the allowed prefixes are feature/, fix/, docs/, refactor/, test/ and chore. The description may contain only letters, numbers, ".", "_" and "-". Examples include feature/work-item-state, docs/ClassDiagramV2.

The prefix is not cosmetic, it determines the label applied to the pull request automatically, and the CI (continuous integration) workflow that validates branch names will reject a branch that does not follow the convention. This keeps the branch list, the PR list and the label list consistent without manual bookkeeping.

Commit and Pull-Request Conventions

Commit messages and pull-request titles both follow the form:

type: what changed

Where the allowed types are feat, fix, docs, refactor, test and chore. feat corresponds to the feature/ branch prefix, the other types match their branch prefixes directly. E.g. docs: update class diagram with Visitor

Pull-Request Process

Every branch is opened as a pull request against main. The repository has a pull-request template that prompts the author to record:

- A one to three sentence summary of the change;
- The type of change;
- The design pattern(s) and classes the change affects
- How the change was tested;
- A checklist covering branch freshness, compilation, the avoidance of large if/else or switch chains, diagram updates, design-decision logging and a self-review of the diff

The template is the mechanism by which the project rules are checked at review time rather than remembered. A reviewer who sees an unchecked box or a missing pattern reference can ask for it directly and the author has already been prompted to supply it.

A pull request is merged only after:

1. Four approvals from teammates other than the author (meaning a total of five people have seen the change and believe it should be merged);
2. All review conversations resolved;
3. All automated checks passing;

4. The branch being up to date with main

Because approvals are dismissed when new commits are pushed, the reviewer effectively re-approves the state of the branch that is actually merged. This prevents a pull request from being approved in one form and merged in another.

Automated Checks

Four GitHub Actions workflows run on every pull request into main:

- build: runs make on ubuntu-latest to confirm the project compiles with C++11;
    - This check is minimal, it confirms compilation, not correctness. Correctness is to be established by testing.
- check-branch-name: validates the branch name follows prefix/description;
- check-title: validates the PR title follows type: description;
- label: applies the label derived from the branch prefix
    - This, unlike the other three, does not block a merge. The others are gating checks and they enforce the conventions at the point of the merge, whilst this is simply advisory.

Merging

Two merge strategies are permitted:

- Squash and merge is used when the branch contains many small, low-value commits or when the change is a single coherent idea. The pull-request title becomes the single commit message, which is why the title is enforced.
- Merge commit is used when the branch contains commits that each carry meaningful design information and are worth preserving in the history.
    - When a merge commit is used, the default subject line is advisably edited to state what the pull request did, e.g. feat: add WorkItem state hierarchy (#12). This ensures that merge commits are readable in git log and in the GitHub history view.

After merging, the branch is deleted. The pull request remains as a record of the change, including the review conversation, the automated check results and the final approved diff.

Keeping Branches Current

When main advances while a branch is in progress, the author brings the branch up to date before requesting re-review:

```
git checkout main
git pull
```

git checkout feature/work-item-state
git merge main
git push

git rebase main followed by git push --force-with-lease is permitted for branches that only one member is working on, and produces a cleaner history. Rebase is not used on branches that another member is actively working on, and the team is aware that force-pushing dismisses existing approvals.

Overall, this workflow supports the project rules. The design decision log, team contribution statement and revision history will be maintained throughout the project, rather than retrospectively at the end of the project. The log is a genuine record of the design process rather than a summary written after the fact.

Team Contribution Statement

Aidan
- Scenario definition (employee onboarding)
- Scenario-specific activity diagram

Connor
- Role designation & task cycling
- Generic workflow execution activity diagram
- cross-diagram and design consistency review

Ishmael
- Pattern mapping & justifications
- Tracking of design decisions

Jay
- Review of written work (incl. formatting, typo correction, in-text citations, etc.)
- Documentation management
- Sequence diagrams
- Design rationale
- GitHub workflow setup and CONTRIBUTING.md

Keval
- Research
- Proposed system scope

Ruan
- Consolidation of class diagram

Thabo

- Activity diagram
- State diagram